Voting Mechanisms in Multi-Agent Economic Simulations

Project Synopsis

Shri Hari S — Roll No. 261090830005

Introduction

Most economic policy design still leans on one of two things: simplified mathematical models, or historical data pulled from the real world. Neither holds up well once you try to account for how people actually behave — human decision-making, political pressure, and individual preference all shift constantly, and static models just aren't built to track that. Multi-agent reinforcement learning (MARL) offers a different route. Instead of solving an economy top-down, it builds one from the bottom up: autonomous agents learn to work, consume, and trade, while a separate planning system decides on tax policy and public spending.

The catch is that frameworks like Salesforce's AI Economist still treat the planner as a black box that answers to no one. There's no vote, no election, no accountability loop of any kind — the planner just optimizes. Real economies don't work that way. Citizens vote, they elect people based on what matters to them financially, and policy is (at least in theory) shaped by that feedback. This project takes the AI Economist's Foundation framework and bolts an endogenous voting mechanism onto it — candidate leaders run on tax proposals, worker agents periodically vote, and we get to watch how that democratic loop changes wealth distribution, how stable policy stays over time, and how healthy the resulting economy actually is.

Problem Statement

Reinforcement-learning approaches to economic policy generally treat the central planner as a free agent chasing a fixed social welfare function, with nothing constraining it politically. That framing misses a few things that matter in practice:

- Centralized planners in these simulations answer to no one — there's no accountability mechanism tying policy back to the agents living under it.
- None of the existing models aggregate what individual voters actually want, so a policy can look optimal on paper and still leave most agents dissatisfied.
- Almost nothing has been done on how economic behavior (labor, consumption) and political behavior (voting) evolve together when agents are learning both at once.
- Skip the voting mechanism entirely, and a simulation can't capture the feedback loop that real economies run on — citizen satisfaction shaping policy, policy shaping productivity, and productivity shaping satisfaction again.

Objectives

1. **Build the voting mechanism**
   Add a periodic election module inside the AI Economist Foundation environment that can handle candidate tax proposals and let worker agents cast ballots.

2. **Define the reward and utility functions**
   Work out multi-agent reward structures that balance what workers care about (wealth and consumption, minus the cost of labor) against what candidate leaders care about (winning elections, and the economic output on their watch).

3. **Train the multi-agent models**
   Train worker policies (labor decisions plus voting choices) and leader policies (policy proposals) at the same time, using multi-agent RL algorithms.

4. **Evaluate the outcomes**
   Compare policy stability, wealth distribution, voter satisfaction, and overall economic performance between the democratically-elected runs and the standard central-planner baseline.

Evaluated Outcomes

The analysis looks at how democratic decision-making reshapes system behavior, split across two broad groups of metrics.

Economic indicators

Overall productivity (GDP), wealth inequality via the Gini coefficient, how easily agents move between income tiers, how efficiently resources get used, tax revenue generated, and a few related economic measures.

Political and social dynamics

Voter satisfaction, how often policy turns over and how volatile tax rates get, how often incumbents get re-elected, whether voter preferences cluster into distinct groups, and broader social-welfare indicators.

Requirements

Hardware

- **Processor:** multi-core CPU, 6 cores minimum (e.g. AMD Ryzen 7 or Intel i7)
- **Memory:** 16 GB RAM
- **Graphics:** NVIDIA GPU, or cloud GPU access via Google Colab
- **Storage:** at least 15 GB free SSD space

Software

- **Containerization:** Docker
- **Simulation engine:** AI Economist (Foundation framework)
- **Language:** Python 3.10+
- **Frameworks:** PyTorch, PettingZoo / Gymnasium
- **Data processing & visualization:** NumPy, Pandas, Matplotlib

Timeline (8 Weeks)

Week	Focus Area	Deliverables & Tasks
Wk 1	Environment setup	Get the Docker container running and make sure the base AI Economist environment steps through without errors before touching anything new.
Wk 2	Voting architecture	Build out the custom voting component and wire it into the environment's observation and action spaces.
Wk 3	Election logic	Add periodic election triggers, candidate tax-proposal mechanics, and the ballot-processing logic.
Wk 4	Reward formulation	Write and test the utility functions for workers and for candidate planners.
Wk 5	Policy setup	Set up the multi-agent training pipeline and run a first round of baseline experiments.
Wk 6	Joint training	Train the economic and political layers together and tune hyperparameters.
Wk 7	Experiments & logging	Run the comparative experiments and log economic and political metrics as they come in.
Wk 8	Documentation	Clean up the repository, generate the final plots, and pull the report together.